Final Project Report

Deliverable 3, Campus Ride-Pooling and Split-Fare System

Prepared by Mainak Sarkar and Khushi Ranawat

Prithviraj Ghosh (230794) Mainak Sarkar (230619) Khushi Ranawat (230562) Mahir Jain (230618) Chaitanya Nitawe (230315)

Contents

1 Overview and final scope 1

2 Jira setup and dashboard 2

3 Sprint execution 3

4 GitHub workflow 3

5 Traceability 4

6 Quality summary 4

7 Defect tracking 5

8 Deployment, timeline and cost 5

9 Individual contributions 6

10 Gaps and lessons 7

1. Overview and final scope

The Campus Ride-Pooling and Split-Fare System is a web application for IIT Kanpur students travelling between campus and transport hubs in the city. A student offers a ride or requests a seat on someone else's ride, the fare is shared equally among the people in the car, and the group coordinates through a private chat. The application calculates who owes what but does not handle money: settlement happens outside it, by cash or UPI, and is only recorded in the app.

The system runs at `https://campus-ridepooling-automation.onrender.com/`.

1.1 User roles

- **Student.** One account does both jobs: a student can be the *ride owner* of one ride and a *rider* on another. Accounts are limited to verified IITK email addresses.
- **Operations administrator.** Manages hubs, pickup points, vehicle types and fare estimates, reviews complaints and SOS incidents, and issues warnings or suspensions. Students cannot open administrator routes, and administrator actions are recorded in an audit log.

1.2 Main workflow

1. A ride owner publishes a route, departure window, vehicle type, capacity and expected fare.
2. Riders search the eligible rides and request a seat. The owner accepts or rejects each request.
3. The ride locks before departure, which fixes each accepted rider's share of the fare.
4. The group coordinates in the ride chat, checks in, and records arrivals and any no-shows during the trip. Occupants can raise an SOS alert.
5. When the trip completes, the system stores the final shares. Riders record their settlement, rate each other and can file a complaint, which an administrator reviews.

Seats are the shared resource: several riders can compete for the last seat at the same moment, and no seat may be allocated twice. Row locks in the request workflow and capacity triggers in the database enforce this, and the concurrency report tests it.

1.3 AI features

Gemini is used in three places, and in each one a person or a validated rule keeps the final decision.

- **Ride recommendations** rank search results that have already been checked as eligible.
- **The chatbot** interprets natural-language requests and can only call allow-listed server operations. Changing a saved preference needs the rider's explicit confirmation.
- **Complaint analysis** gives an administrator advisory input. The administrator makes any enforcement decision.

Each feature has its own fallback if Gemini is unavailable or returns unusable output. Ride recommendations rank the eligible rides by departure timing and fare. The chatbot interprets the request with deterministic rules where it can, and links the rider to manual search when it cannot answer. Complaint analysis records that no AI result is available, and the administrator reviews the complaint manually. The failure and recovery report covers the recommendation and chatbot cases.

1.4 Technology

The application is written in TypeScript with Next.js 16. A custom Node.js server hosts Socket.IO for live ride and chat updates, and PostgreSQL 16 stores all data. Tests use Vitest and Playwright, and GitHub Actions runs them on every pull request. It is packaged with Docker and deployed on Render.com (Section 8).

1.5 Final scope

The requirements and design are in the Deliverable 1 documents, and the implementation baseline, API documentation and core workflow demonstration are in Deliverable 2. The final traceability matrix lists the 194 Deliverable 1 requirements that a passing automated test verifies.

One decision changed after Deliverable 1: the deployment platform moved from GCP to Render.com. The submitted requirement is unchanged, and the change is recorded in the deployment documentation (Section 8).

2. Jira setup and dashboard

The team used Jira to organise the Campus Ride-Pooling and Split-Fare System under the project key `CS455` (project name: *CS 455 Car-pooling Project*). The project is a Jira software project. Work was organised as a hierarchy of larger deliverables and user stories, with smaller implementation tasks tracked separately. The issue types used in the project include Epics, Stories, Tasks, Bugs and Subtasks. Subtasks break larger pieces of work into smaller, assignable units; defects are recorded as Bug issues.

The board provides the team's shared view of the backlog and sprint work. Issues move through the working statuses *To Do*, *In Progress*, *In Review* and *Done*. Issues can be assigned to a team member, prioritised and associated with a sprint. Sprint records also hold the sprint goal and dates where these were configured.

The four named iterations represented in Jira are *Deliverable 0*, *Sprint 3 (D2)*, *Sprint 3 (D2) 2* and *Sprint 3 (D3)*. Not all project work was consistently assigned to a sprint.

Jira was used alongside GitHub. Issues can be associated with implementation evidence in the repository, including branches, commits and pull requests. This provides a path from a tracked task to the code change and its review. The defect records in Section 7, for example, link back to the bug-evidence tracking story and identify the pull requests responsible for the fix and its tests.

The project dashboard provides a consolidated view of progress from Jira issues and the board. It supports a summary-level review of work distribution and backlog state, while the board and individual issues provide the assignment, sprint and development-link details behind that summary.

Table 1. Jira issues by type and status, from the Jira snapshot of 6 Nov.

Type	Total	Done	In Progress	In Review	To Do
Epic	2	2	0	0	0
Story	28	28	0	0	0
Task	13	13	0	0	0
Subtask	101	101	0	0	0
Bug	29	29	0	0	0

The counts are generated from the Jira data used by the report rather than typed into this section. They describe the snapshot date in the table's caption and are not live counts after that date.

3. Sprint execution

The team used Jira sprints to group a portion of the project work into time-bounded iterations. The four sprints represented in Jira are *Deliverable 0*, *Sprint 3 (D2)*, *Sprint 3 (D2) 2* and *Sprint 3 (D3)*. Their recorded start and planned end dates, issue counts and completion-by-close counts are summarised in Table 2. Done by close means that an issue's Jira resolution date is on or before the sprint's close date; it is not simply the issue's current status.

Table 2. Sprints recorded in Jira.

Sprint	Started	Planned end	Closed	Issues	Done by close
Deliverable 0	26 Aug	29 Aug	1 Oct	6	6
Sprint 3 (D2)	2 Oct	8 Oct	4 Oct	7	7
Sprint 3 (D2) 2	4 Oct	10 Oct	28 Oct	88	88
Sprint 3 (D3)	28 Oct	7 Nov	6 Nov	44	44

Deliverable 0. The recorded goal was to prepare the one-page project proposal. It covered the project description and user roles, the core transactional workflow, the shared resource and concurrency challenge, the intended use of AI agents and their tools, the proposed technology stack, initial risks and technical uncertainties, and the initial Jira/GitHub development strategy.

Sprint 3 (D2). The recorded goal is *Web app scaffolded and sign-in/registration working end to end.* The sprint closed on 4 October, four days before its planned end, with all of its issues done. The table gives the issue and completion counts from Jira; it is not a complete measure of all D2 work, since some work was not assigned to a sprint.

Sprint 3 (D2) 2. This sprint started as soon as the first D2 sprint closed, and it has no written goal in its Jira goal field, so its scope is best understood from the issues assigned to it. It contains further D2 implementation and refinement work. It stayed open until 28 October, 18 days past its planned end. Assigned issues and their recorded resolution dates provide the basis for the completion figures. This second sprint record is retained separately, as it appears in Jira.

Sprint 3 (D3). The recorded goal is *Deliverable 3.* Its issues cover the integration, adversarial, performance, system, failure and security testing, the defect history, the deployment, the timeline, the costing, the traceability matrix, and the final dashboard and report. It closed on 6 November, the day of the deadline, and 14 of its issues were resolved that day.

Planning and review limitations. Sprint planning was not applied consistently across the entire project. Some issues were left without a sprint assignment, so work performed outside the four recorded sprints is not reflected in the sprint table. Story-point estimates were also incomplete: 3 of 173 issues have one, so the team cannot use the Jira records to make a reliable effort-based velocity comparison. Issue counts are not proxies for equal effort.

Every issue in all four sprints was resolved by the sprint's close, so the records show no work carried forward. That does not mean the work finished on plan: Deliverable 0 closed on 1 October against a planned 29 August, and Sprint 3 (D2) 2 closed on 28 October against a planned 10 October.

The Jira sprint records do not provide a consistent written record of sprint reviews or retrospectives for every iteration. Accordingly, this section does not claim that a review or retrospective took place where there is no recorded evidence. The table and descriptions are limited to sprint metadata and issue history available in Jira.

4. GitHub workflow

All development is in the team repository, `CS455-Carpooling/Campus_Ridepooling_Automation`, with each member working from their own account. Work is meant to follow one path: a branch named with its Jira key, commits carrying that key, and a pull request into `master` using the template in `.github/PULL_REQUEST_TEMPLATE.md`, which asks for the Jira issue, the requirements addressed and how the change was tested.

GitHub Actions runs three jobs on every pull request: format, lint, type-check, unit tests with the 80 % coverage gate and a production build; integration tests against PostgreSQL; and Playwright system tests.

Table 3 shows how closely the team followed this path, from a snapshot of `master` at `5c8411d` on 6 Nov. Of 98 merged pull requests, 30 have a recorded review from another member and 77 name a Jira key in their title, branch, commits or Jira section. Members submitted 37 reviews in total, of which 6 requested changes. Reviews were expected but not enforced, because `master` has no branch protection, and many pull requests were merged without one (Section 10).

Table 3. Merged pull requests by deliverable, using the merge date in IST.

Phase	Merged	Reviewed by another member	Approved	Names a Jira key
D0 and D1 (to 26 Sep)	14	0	0	8
D2 (27 Sep to 15 Oct)	63	18	14	54
D3 (from 16 Oct)	21	12	5	15
Total	**98**	**30**	**19**	**77**

A review here means a submitted GitHub review. Comments left on a pull request outside a review are not counted, so the table can understate informal review.

5. Traceability

The requirements traceability matrix links 194 Deliverable 1 requirements to 80 Jira issues, 51 merged pull requests and 303 automated tests and test groups. It is generated from recorded test runs and a snapshot of the pull requests, so it can be rebuilt and checked rather than maintained by hand. The full table is also available as `Deliverables/D3/Traceability_Matrix/traceability-matrix.csv`.

Table 4 follows one requirement along the whole chain. FR-RD-06.8 says that once a rider is accepted on a ride, their other active requests for rides departing within two hours are withdrawn and those owners are told.

Table 4. One complete chain, for requirement FR-RD-06.8.

Requirement	FR-RD-06.8 (Must), in the rider functional requirements
User story	US-RD-18, acceptance criterion US-RD-18-AC2, in the rider user stories
Jira issue	CS455-76 (implementation, under story CS455-75) and CS455-111 (tests, under story CS455-109)
Branch	`CS455-76-CS455-77-Ride-Lifecycle` and `CS455-111-core-workflow-tests`
Commits	6b1746b (implementation) and 6b258c5 (tests)
Pull request	#55 (implementation, reviewed by Mainak Sarkar) and #85 (tests)
Test case	“withdraws a rider’s other request within two hours once they get a seat (FR-RD-06.8, P-08)”, in `apps/web/test/integration/rides/seats-and-lock.int.test.ts`
Test result	Passed in the integration run recorded on 2026-11-06, in `Deliverables/D3/test-run/integration-results.json`

Two limits apply to the chain. Implementation and tests were often written by different people at different times, so one requirement can span several Jira issues and pull requests, as it does here. And 21 of the 98 merged pull requests name no Jira key, so the chain cannot start from Jira for those.

6. Quality summary

Table 5 gives the main result of each Deliverable 3 report. The reports hold the method, evidence and steps to reproduce each result. The code and the tests are the same across them, so their counts overlap and should not be added together.

Table 5. Main result of each Deliverable 3 report.

Report	Owner	Main result
Unit test and coverage	Khushi Ranawat	1544 tests passed. Statement coverage is 87.7 %, above the 80 % gate that continuous integration enforces.
Integration test	Mainak Sarkar	71 of 71 tests passed in 12 files against a real PostgreSQL 16 database, 9 of them regression tests for defects the suite found.

Report	Owner	Main result
Adversarial test	Mainak Sarkar	41 of 41 cases passed in 4 files after a fix. The suite found one real weakness, an idempotency key replaying the wrong request, fixed in #87.
System test	Khushi Ranawat	48 of 48 browser tests passed in 13 files against the complete stack, with no backend mocks.
Concurrency	Khushi Ranawat	5,000 mixed route-handler operations in 100 rounds on one ride, with zero capacity violations.
Performance and load	Mahir Jain	At 100 virtual users, a SQL optimisation lowered p95 latency from 2.27 s to 1.76 s (22.5 %). Reliability and correctness targets passed. The p95 and p99 latency targets were not met.
Security and threat model	Chaitanya Nitawe, Prithviraj Ghosh	Six threats modelled with STRIDE and seven security defects fixed. In the report's own run, 61 focused unit tests and the 110 PostgreSQL integration tests passed.
Failure and recovery	Chaitanya Nitawe, Prithviraj Ghosh	Five failure domains covered, including LLM timeout and duplicate requests. In the report's own run, 61 focused unit tests and the 110 PostgreSQL integration tests passed.
Bug and defect history	Khushi Ranawat	26 resolved application defects, each with its Jira Bug and pull request (Section 7).
Traceability matrix	Mainak Sarkar	194 requirements traced to 303 passing tests and test groups.

7. Defect tracking

Defects found during development were tracked in Jira using the *Bug* issue type. 26 Bug issues carry the `d3` label: 16 from the Deliverable 3 security, reliability and database audit, under the bug-evidence tracking story CS455-121, and 10 product defects from the review of Deliverable 1, under CS455-86. The bug records capture the observed failure, affected behaviour, related project work item and implementation evidence. The other three Bug issues (CS455-4, CS455-5 and CS455-6) are Deliverable 1 documents that were filed as Bugs.

All of these issues are marked Done. The 26 defect records were created after the corresponding fixes had already been implemented. They are retrospective defect-history entries, not evidence that the fixes were first made in response to newly opened Jira tickets.

Two of the defects show how a fix and its evidence can sit in separate pull requests:

- CS455-137: realtime authentication could accept a session after the seven-day inactivity period, even though HTTP authentication rejected it.
- CS455-141: malformed percent-encoded cookies could cause an exception during WebSocket authentication.

For both defects, #91 contains the code fix and #94 added the regression tests. The regression coverage is in `apps/web/test/server-security.test.ts`. It checks rejection of sessions outside the inactivity window and malformed-cookie handling during WebSocket authentication.

The detailed defect descriptions and evidence are maintained separately in the repository's D3 Bug Report. This section summarises Jira tracking and the relationship between the issues and pull requests rather than duplicating the full bug report.

8. Deployment, timeline and cost

8.1 Deployment

The application runs on Render.com's Free tier, as a Docker web service with a Render managed PostgreSQL database. Pushes to `master` deploy automatically, and migrations run when the container starts. The health endpoint is `https://campus-ridepooling-automation.onrender.com/api/health`. The steps, configuration and verification are in the deployment documentation.

Deliverable 1 planned GCP (requirement NFR-RD-18). The team chose Render on 5 November because it builds the repository's Docker image directly from GitHub, provides managed PostgreSQL and runs the custom server that Socket.IO needs. The submitted requirement still says GCP, and the change is recorded in section 7 of the deployment documentation rather than by rewriting Deliverable 1.

The Free tier has limits that matter for the demonstration. The service sleeps after 15 minutes without traffic and takes about a minute to wake. The database expires on 5 December 2026 and has no backups.

8.2 Timeline

The timeline and Gantt chart (CS455-162) covers the project from the proposal to the final demonstration, and Figure 1 on the last page shows it. Each work bar runs from the day the pull requests for that work opened to the day they merged. Deliverable 1 work ran from 23 to 26 September, implementation from 2 to 15 October, and Deliverable 3 testing, reporting and deployment from 28 October to 5 November.

8.3 Cost

The costing report (CS455-178) estimates the monthly operating cost of the current setup and of two production sizes, from prices checked on 6 November 2026 at a rate of 94 rupees to the dollar. Table 6 summarises it. Totals include 18 % GST.

Table 6. Estimated monthly operating cost.

Setup	Web service	Database	Monthly cost
Current demonstration	Render Free	Render Free, 1 GB	INR 0
Pilot, about 1,000 students	Render Starter	Basic-256mb, 5 GB	INR 2,917 (USD 31.03)
Full scale, about 5,000 students	Render Standard	Basic-1gb, 20 GB	INR 13,954 (USD 148.44)

Model inference is the largest cost at both production sizes: INR 1,034 of the INR 2,472 pre-tax pilot cost, and INR 5,170 of INR 11,825 at full scale. That figure assumes 20,000 and 100,000 Gemini calls a month, at 1,000 input and 200 output tokens per call. The input figure comes from the requests the code really builds; the output figure is not measured, so the costing report also gives the effect of doubling it. Salaries, monitoring tools and bandwidth above the included allowance are excluded.

The current setup stays within its free limits. A single web service stays within Render's 750 free hours a month, a freshly migrated database is about 11 MB of its 1 GB limit, and the database expires after the presentation window. GitHub Actions ran 468 workflows, about 1,040 elapsed minutes, at no charge, because standard runners are free for a public repository. Section 7 of the costing report compares estimated and actual usage and lists the production figures that were not collected.

Section 9 of the costing report gives an AI-assisted estimate of development time, made after the work: 181 member-days from the size of what was built, against 61 member-days with a commit. Commit days undercount effort and no timesheets were kept, so the true figure lies somewhere between the two.

9. Individual contributions

Table 7 combines each member's GitHub and Jira activity, from the snapshots of 6 Nov (GitHub) and 6 Nov (Jira). These are counts of recorded activity and not a measure of effort: one commit can be a single line or a whole feature, and Jira assignment reflects how each member broke down their work.

Table 7. Recorded activity per member. Commits exclude merge commits; reviews count the distinct pull requests a member reviewed.

	GitHub			Jira issues		
Member	Merged PRs	Commits	PRs reviewed	Assigned	Done	Reported
Prithviraj Ghosh	4	24	2	14	14	8
Mainak Sarkar	40	112	24	58	58	56
Khushi Ranawat	11	185	4	20	20	44
Mahir Jain	33	93	1	57	57	41
Chaitanya Nitawe	10	73	0	21	21	24

Table 8. Main areas of work per member, with the pull requests that hold them.

Member	Main areas
Prithviraj Ghosh	First version of the Ride Owner requirements; authentication pages (#19); chatbot (#75); security and failure reports with Chaitanya Nitawe (#94).

Member	Main areas
Mainak Sarkar	Rider requirements (#1); application scaffold and landing page (#16, #17, #18); ride inputs, details and my rides (#21, #27 to #31); ratings (#36 to #40); operations admin foundation, configuration and complaints (#42 to #44, #47 to #49); trip and SOS (#56, #57); D1 bug review (#67); integration and adversarial suites and reports (#83 to #89); traceability matrix (#93).
Khushi Ranawat	Operations Admin requirements (#3); create-ride flow (#24); AI ride recommendation and hobby tags (#51, #52); notifications (#53, #54, #61, #66); unit test report (#92); defect history (#96); concurrency experiment and seat-capacity guard (#97); system test report.
Mahir Jain	Class, sequence and architecture diagrams and design patterns (#10, #11, #14); authentication backend (#20); profile and history (#33); pool chat (#41); AI review panel (#50); ride lifecycle (#55); hardening, UI and fares (#69 to #74); LLM integration (#76); D2 documentation (#77 to #80); performance testing (#90); audit defect fixes (#91); deployment (#98); timeline and Gantt chart (#99, #103).
Chaitanya Nitawe	Ride Owner requirements (#4 to #6); architecture alternatives and rejected decisions (#12); ride search (#35, #45); ride requests (#62); system test suite (#95); security and failure reports with Prithviraj Ghosh (#94); project costing report (#104).

10. Gaps and lessons

This section records where the project fell short of its own process. The final Jira cleanup on 5 November set statuses and assignees only; the estimates and sprints of past issues were left as they were.

10.1 Jira

- **Estimates.** 3 of 173 issues have a story point estimate, so the team planned and tracked sprints by issue count rather than by estimated effort.
- **Sprints and fields.** 28 issues are in no sprint, 127 have no labels and 3 have no assignee.
- **Statuses lagged behind the work.** 49 issues were marked resolved on 5 November, many of them for work merged earlier, so resolution dates are not a reliable record of when work finished.
- **Issues recorded late.** 42 issues were created on 5 November. They include 24 of the 26 defect Bugs, which were recorded after the defects were fixed rather than when they were found.

10.2 Code review

68 of the 98 merged pull requests have no recorded review from another member, although the handout expects a review on significant pull requests. Nothing enforced it, because `master` has no branch protection.

10.3 Schedule and scope

- **Late start on testing.** `master` has no commits for 12 days between 15 Oct and 28 Oct, and Deliverable 3 testing started only at the end of that gap.
- **Late deployment.** The platform was chosen and deployed on 5 November, the day before the deadline, which left little time to run the system under real conditions.
- **Performance targets.** The p95 and p99 latency targets were not met, and the performance report states what would need rerunning before making a stronger claim.

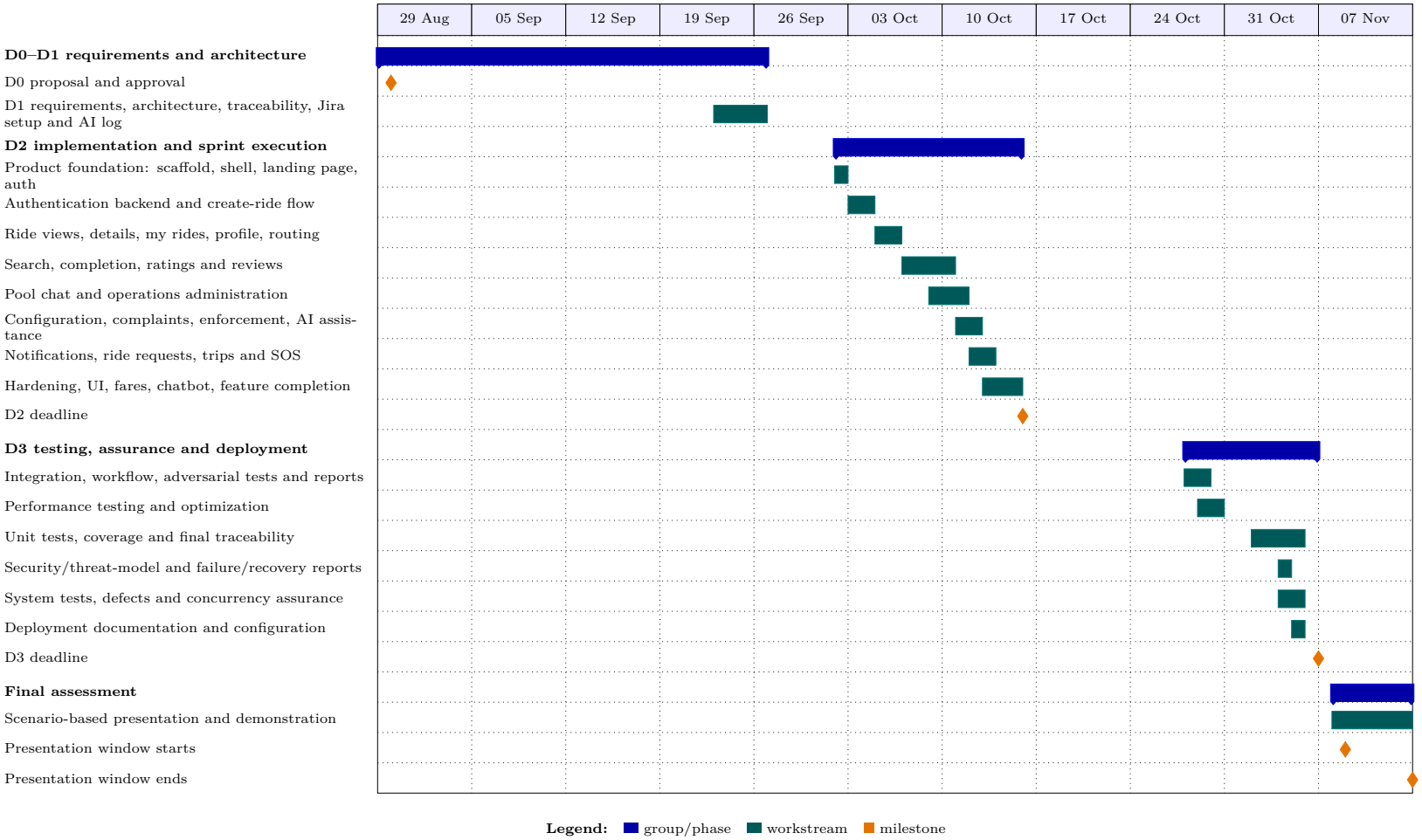
10.4 Lessons

- **Test against the real database early.** The unit suite cleared its 80 % coverage gate with the database mocked, yet the first real PostgreSQL runs found defects it could not see. The coverage gate is necessary but does not show that the parts work together.
- **Update Jira as the work happens.** A status or Bug recorded at the end costs as much time as one recorded on the day and is worth less as evidence.
- **Turn the process into a rule.** Requiring a review through branch protection from the first day would have settled the review gap without relying on anyone remembering.
- **Deploy early.** A walking skeleton on the final platform during Deliverable 2 would have tested the deployment choice while there was still time to change it.

Campus Ridepooling Automation

Project Timeline and Gantt Chart

29 August–13 November 2026



Legend: group/phase workstream milestone

Column headers: each date (for example, "29 Aug") is the first day of a seven-day calendar column; the following six days continue to the right. Thus, "05 Sep" means the week beginning 5 September 2026, not a single-day deadline. Bars show their dated work windows, while diamonds mark milestones.

Figure 1. Project timeline and Gantt chart, from the timeline document.